NovaTech Industries

Checkpoint 2.6 - Synthetic Data Generator, Step 2

Document ID: NT-DATA-CP2.6-S2

Version: 1.0

Date: 21 October 2026

Status: Development exercise

Related requirements: FR-001, FR-002, FR-005; Data Dictionary v1.0

> **Scenario note:** NovaTech Industries is a fictional company and all records generated in this exercise are synthetic.

1. Purpose and scope

Expand the verified three-table sample from Checkpoint 2.6 Step 1 into a larger, reproducible reference dataset. Keep the existing `data/synthetic/v1/` unchanged and write the new output into `data/synthetic/v2/`.

| Output file | Expected records | Description |

|---|---:|---|

| `suppliers.csv` | 12 | Supplier identifiers, names, countries and statuses |

| `products.csv` | 20 | Product identifiers, SKUs, descriptions and categories |

| `supplier_products.csv` | 45 | Valid supplier-to-product relationships and quantities |

| **Total** | **77** | **Synthetic reference records** |

Not yet included: quotations, prices, lead times, quality history, exchange rates, and scoring. These will be introduced in subsequent steps; no supplier recommendation can be calculated from these three tables alone.

2. Learning objectives

1. Use a **random seed** to reproduce the same generated dataset on every run.
2. Use **primary keys** (`supplier_id`, `product_id`) to identify unique records.
3. Use **foreign keys** in `supplier_products` to reference existing records.
4. Prevent duplicate supplier-product pairs with a **composite key**.
5. Use **assertions** to detect invalid data before saving files.
6. Use `pathlib` and `csv` from the Python standard library; no package installation is required.

3. Project directories

From your existing project root, create only the missing directory:

```
cd ~/Documents/Projects/ai-procurement-platform
mkdir -p data/synthetic/v2
```

Create a **new** Python file rather than replacing the working v1 generator:

```
ai-procurement-platform/
■■■ data/
■   ■■■ synthetic/
■       ■■■ v1/                  # Existing, preserved
■       ■■■ v2/                  # New CSV output
■■■ src/
    ■■■ data_generation/
        ■■■ generate_reference_data.py
        ■■■ generate_reference_data_v2.py
```

4. Python implementation

Copy the complete code below into `src/data_generation/generate_reference_data_v2.py`.

```
"""NovaTech Industries - Checkpoint 2.6, Step 2.
Generate reproducible, related synthetic reference datasets.
Run from the project root:
    python3 src/data_generation/generate_reference_data_v2.py
"""

import csv
import random
from pathlib import Path

SEED = 2026
rng = random.Random(SEED)
PROJECT_ROOT = Path(__file__).resolve().parents[2]
OUTPUT_DIR = PROJECT_ROOT / "data" / "synthetic" / "v2"

SUPPLIER_NAMES = [
    "EuroParts GmbH", "Shenzhen Components", "Nordic Industrial AB",
    "GlobalTech Mexico", "Delta Controls BV", "Alpine Systems AG",
    "Baltic Automation OÜ", "Pacific Circuit Ltd", "Iberian Drives SL",
    "Rhein Mechanical GmbH", "Orion Electronics Inc", "Polaris Supply AS",
]
COUNTRIES = [
    "Germany", "China", "Sweden", "Mexico", "Netherlands", "Switzerland",
    "Estonia", "Taiwan", "Spain", "Germany", "United States", "Norway",
]
CATEGORIES = ["Sensors", "Motors", "Controllers", "Connectors"]


def write_csv(filename, fieldnames, rows):
    path = OUTPUT_DIR / filename
    with path.open("w", newline="", encoding="utf-8") as file:
        writer = csv.DictWriter(file, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(rows)
    print(f"Created {filename}: {len(rows)} records")


def generate():
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    suppliers = [
        {
            "supplier_id": f"SUP{i:03d}",
            "supplier_name": name,
            "country": COUNTRIES[i - 1],
            "status": "ACTIVE" if i <= 10 else "INACTIVE",
        }
        for i, name in enumerate(SUPPLIER_NAMES, start=1)
    ]

    products = [
        {
            "product_id": f"PRD{i:03d}",
            "sku": f"NT-{i:04d}",
            "product_name": f"Industrial Component {i:02d}",
            "category": CATEGORIES[(i - 1) % len(CATEGORIES)],
            "unit_of_measure": "EA",
        }
        for i in range(1, 21)
    ]

    # Guarantee every product is linked to at least one supplier.
    pairs = {(i % 12, i) for i in range(20)}
    all_pairs = [(s, p) for s in range(12) for p in range(20)]
    remaining = [pair for pair in all_pairs if pair not in pairs]
    pairs.update(rng.sample(remaining, 45 - len(pairs)))

    supplier_products = []
    for supplier_index, product_index in sorted(pairs):
        supplier_products.append({
            "supplier_id": suppliers[supplier_index]["supplier_id"],
            "product_id": products[product_index]["product_id"],
            "minimum_order_quantity": rng.choice([1, 5, 10, 20]),
            "available_quantity": rng.randint(20, 500),
        })

    # Validate data BEFORE writing it to disk.
    supplier_ids = {row["supplier_id"] for row in suppliers}
    product_ids = {row["product_id"] for row in products}
    relationships = {
        (row["supplier_id"], row["product_id"])
        for row in supplier_products
```

```
    }
    assert len(suppliers) == len(supplier_ids) == 12
    assert len(products) == len(product_ids) == 20
    assert len(supplier_products) == len(relationships) == 45
    assert all(row["supplier_id"] in supplier_ids for row in supplier_products)
    assert all(row["product_id"] in product_ids for row in supplier_products)
    assert all(row["minimum_order_quantity"] > 0 for row in supplier_products)
    assert all(row["available_quantity"] >= 0 for row in supplier_products)
    assert {row["product_id"] for row in supplier_products} == product_ids

    write_csv("suppliers.csv", list(suppliers[0]), suppliers)
    write_csv("products.csv", list(products[0]), products)
    write_csv("supplier_products.csv", list(supplier_products[0]), supplier_products)
    print("Validation passed: unique IDs, valid relationships, valid quantities.")
    print(f"Reproducible generation seed: {SEED}")


if __name__ == "__main__":
    generate()
```

5. Run the generator

In the VS Code integrated terminal, from the project root:

```
python3 src/data_generation/generate_reference_data_v2.py
```

Expected output:

```
Created suppliers.csv: 12 records
Created products.csv: 20 records
Created supplier_products.csv: 45 records
Validation passed: unique IDs, valid relationships, valid quantities.
Reproducible generation seed: 2026
```

Inspect the first five lines of each output file:

```
head -n 5 data/synthetic/v2/suppliers.csv
head -n 5 data/synthetic/v2/products.csv
head -n 5 data/synthetic/v2/supplier_products.csv
```

6. Explanation of the key design choices

`rng = random.Random(SEED)` creates a dedicated random-number generator. Re-running the same code with seed `2026` yields the same supplier-product links and quantities. This is useful for debugging and repeatable tests.

`Path(__file__).resolve().parents[2]` locates the project root based on the script's location. This avoids writing CSVs to an unexpected directory when the command is executed elsewhere.

`pairs` **is a set of tuples.** A tuple such as `(0, 4)` represents one supplier-product link. Sets automatically prevent duplicate pairs. The generator first guarantees coverage for all 20 products, then randomly adds unique pairs until there are 45.

`assert` **statements** verify record counts, unique identifiers, valid foreign keys, nonnegative stock, and positive minimum-order quantities. They are suitable for this learning exercise; production applications should use explicit validation and error handling, since Python assertions can be disabled.

Supplier status is not filtered here. Inactive suppliers may remain in reference data. They will be excluded by eligibility rules when evaluating quotations.

7. Data-quality checks and limitations

| Check | Expected result |

|---|---|

| Unique supplier IDs | 12 unique IDs |

| Unique product IDs | 20 unique IDs |

| Unique supplier-product pairs | 45 unique pairs |

| Foreign key validity | Every referenced supplier and product exists |

| Product coverage | All 20 products have at least one supplier link |

| Minimum order quantity | Strictly greater than zero |

| Available quantity | Zero or greater |

| Reproducibility | Identical CSV output on repeated runs with same seed |

This is a **reference-data exercise**, not a complete production data generator. We intentionally use simplified availability quantities and do not model warehouse-specific stock or historical inventory movements yet.

8. Checkpoint exercises

1. Change `SEED` from `2026` to `2027`, run again, and inspect which fields change. Restore `2026` afterward.
2. Find the code that guarantees every product has at least one supplier.
3. Explain why `supplier_products.csv` needs both `supplier_id` and `product_id`.
4. Explain why inactive suppliers are still stored in `suppliers.csv`.
5. Confirm that the original `data/synthetic/v1/` files have not been overwritten.

9. Completion criteria

- [ ] Script is saved at `src/data_generation/generate_reference_data_v2.py`.
- [ ] All three CSVs are created under `data/synthetic/v2/`.
- [ ] Terminal displays the expected counts and validation message.
- [ ] Existing `v1` datasets remain unchanged.
- [ ] You can explain the random seed, foreign key, and composite key concepts.

10. Next checkpoint

Checkpoint 2.6 - Step 3: Add quotations and quotation items, introduce realistic prices and delivery lead times, and begin implementing the approved eligibility rules. Supplier performance history and weighted scoring will follow after their outstanding policies are finalized.

Documentation location: `docs/data/synthetic-data-generator-checkpoint-2.6-part-2.md` and matching PDF.